Engineering Security

Do / Say / See

PR finding, Change Stack, Blast Radius, an AI fix and merge readiness.

5 pillars | 17 core pages | limit 25

The original review is demo-site-pr-examples PR #4. The completed fix is task e6754368 and PR #11 in demo-site-pr-recordings. This guide follows the security video retained unchanged in the Version 3 set.

Read each grouped step top to bottom. Do contains the clicks and scrolls; read only Say aloud. See is a frame from the delivered video. Times refer to that video. The narration is preserved in full, including the recorded opening and closing.

Creation, Send and approval actions describe the recorded flow. Rehearse with the linked existing results; use a separate training run when repeating those actions. Live pages may have changed. Resource links require the corresponding account access.

OPEN THE DEMO

Flagship PR #4

PR #4: Change Stack

PR #4: Security Blast Radius

Completed Engineering Security task

Engineering Security fix: PR #11

PILLARS AND PAGE LIMITS

Review PR Findings - 4 steps / pages

Open Change Stack - 2 steps / pages

Explore Blast Radius - 4 steps / pages

Fix with AI - 4 steps / pages

Review Merge Readiness - 3 steps / pages

The core count includes the opening page when present. This cover, operator prompts and resource directory are outside the core. Every pillar uses at most four grouped steps.

REVIEW PR FINDINGS / STEP 1 OF 4 / 00:01-00:33

Open the PR security finding

DO

Open CodeRabbit Security, then PR security reviews. Filter to coderabbit-demo/demo-site-pr-examples.

Open Unaddressed findings and expand PR #4. Follow its PR/finding link to GitHub and open the invoice-search SQL review comment.

Deep Scan finding: CWE-89

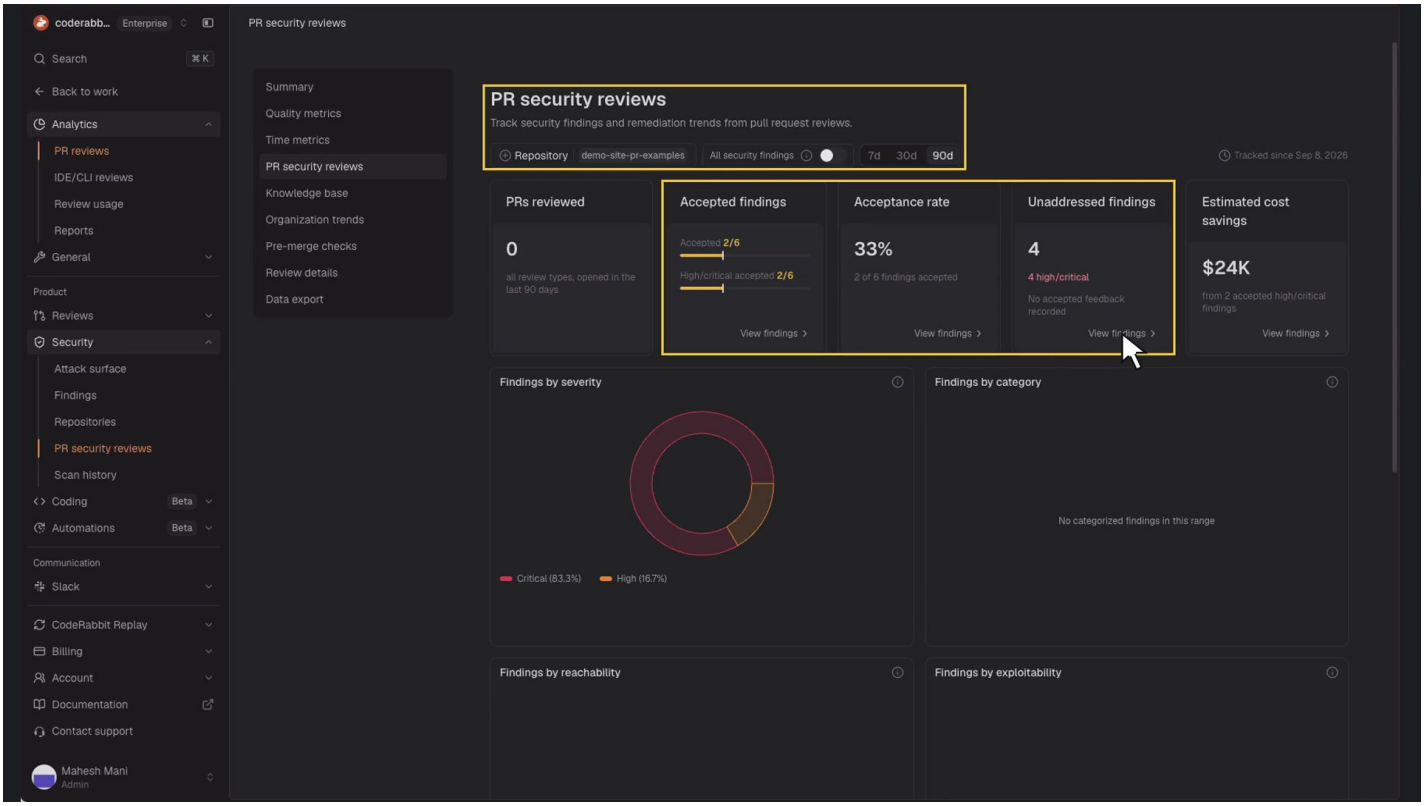
Flagship PR #4

PR #4: SQL finding

SAY

Let's look at security from the engineer side, reviewing a pull request, understanding the impact of a finding, and getting a fix ready for review. We'll start with the security finding in the PR, follow the change through Change Stack and security blast radius, then use fix with AI to prepare a patch. Here in PR security reviews, I'll open pull request 4. This change adds bulk checkout, promotional pricing, and invoice exports. We'll focus on the SQL injection finding in invoice search.

SEE / VIDEO FRAME AT 00:22



REVIEW PR FINDINGS / STEP 2 OF 4 / 00:34-00:48

Inspect the interpolated query

DO

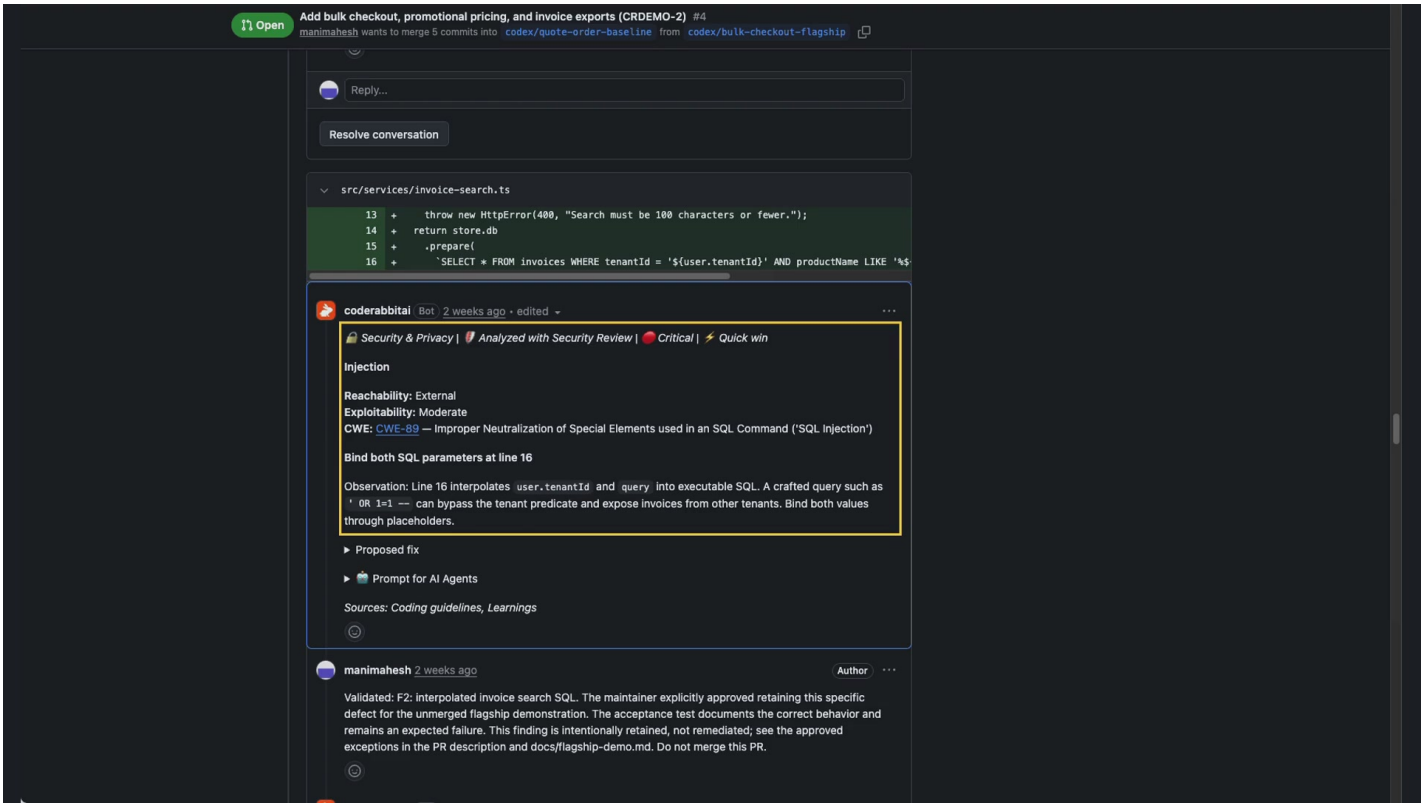
Scroll the finding to the cited invoice-search code.

Show search text and tenant ID inserted into the SQL command. Keep the cross-customer explanation visible.

SAY

The finding points to a query that inserts search text directly into a SQL command. It also inserts the tenant ID into that command. A crafted search could change the query and return invoices belonging to another customer.

SEE / VIDEO FRAME AT 00:43



REVIEW PR FINDINGS / STEP 3 OF 4 / 00:48-00:56

Read severity and reachability

DO

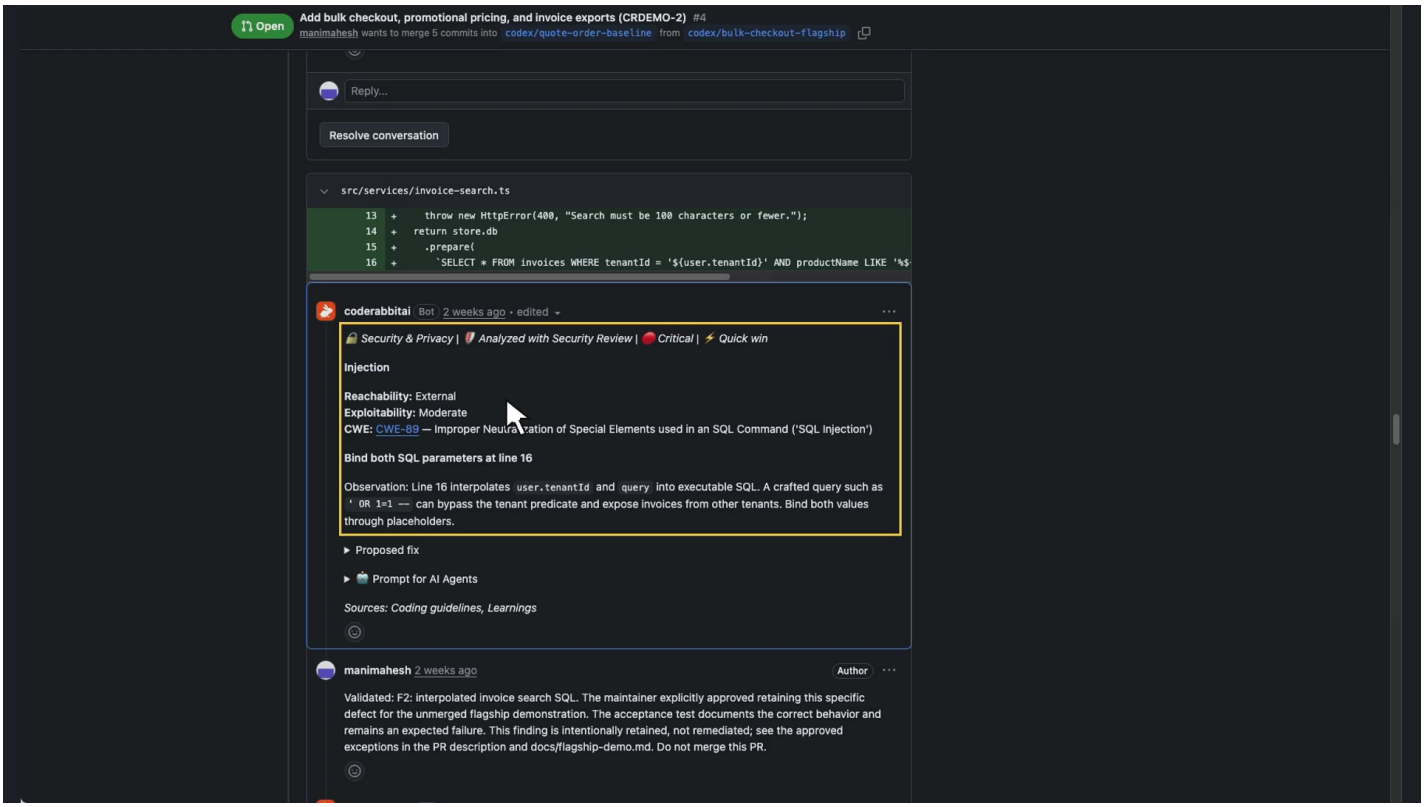
Scroll to the finding summary and read severity, reachability and exploitability.

Keep the affected file and supporting code available.

SAY

I can see the severity, reachability, and exploitability assessment along with the code behind the finding.

SEE / VIDEO FRAME AT 00:54



REVIEW PR FINDINGS / STEP 4 OF 4 / 00:57-01:03

Read the parameter-binding recommendation

DO

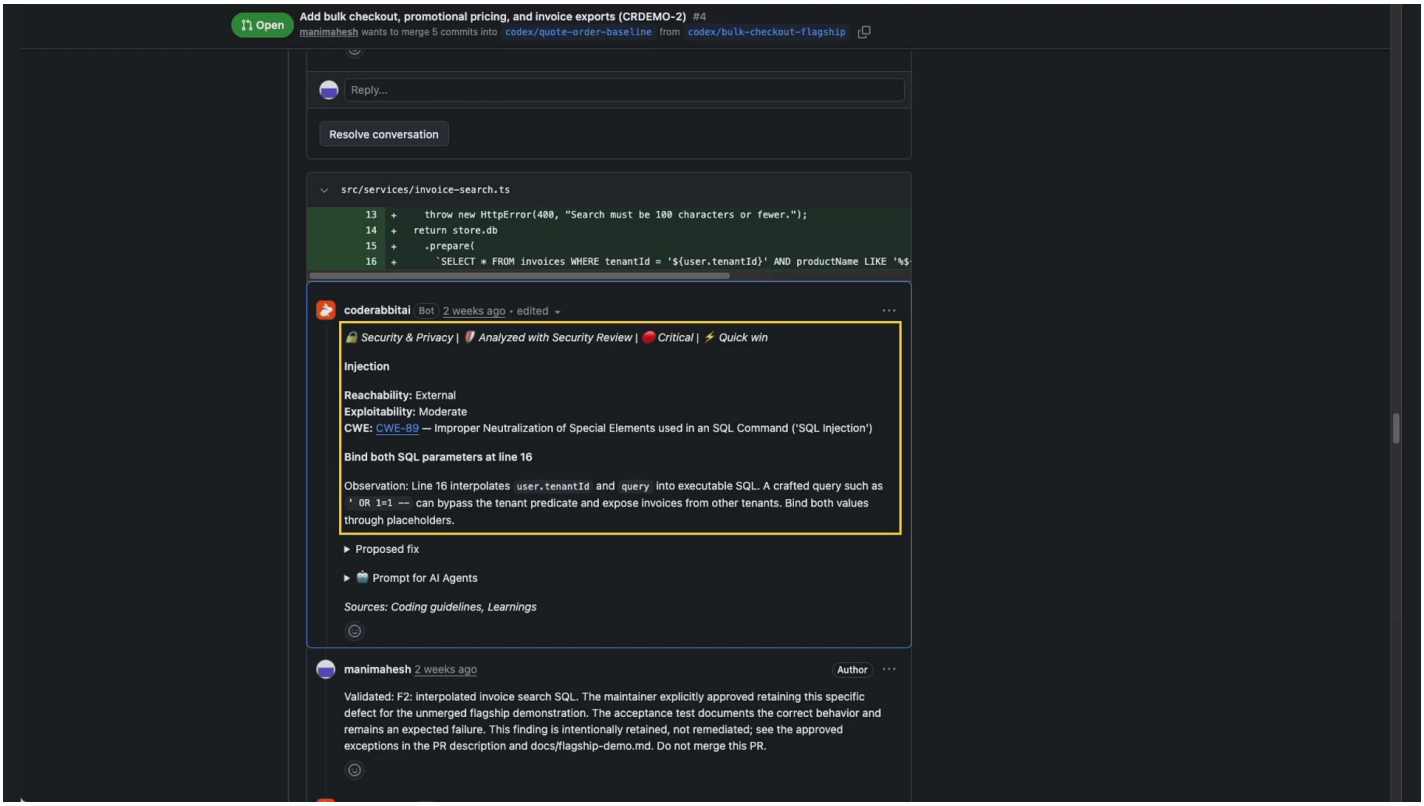
Scroll to Recommendation. Expand the suggested code if collapsed.

Show placeholders with tenant and search values supplied separately.

SAY

The recommended fix is to bind both values as parameters so the database treats them as data.

SEE / VIDEO FRAME AT 01:01



OPEN CHANGE STACK / STEP 1 OF 2 / 01:04-01:11

Follow the PR to Change Stack

DO

Click the PR #4 link and switch to GitHub. Open the SQL review comment.

Scroll to CodeRabbit's walkthrough and click Review in Change Stack. Switch to the new tab.

Flagship PR #4

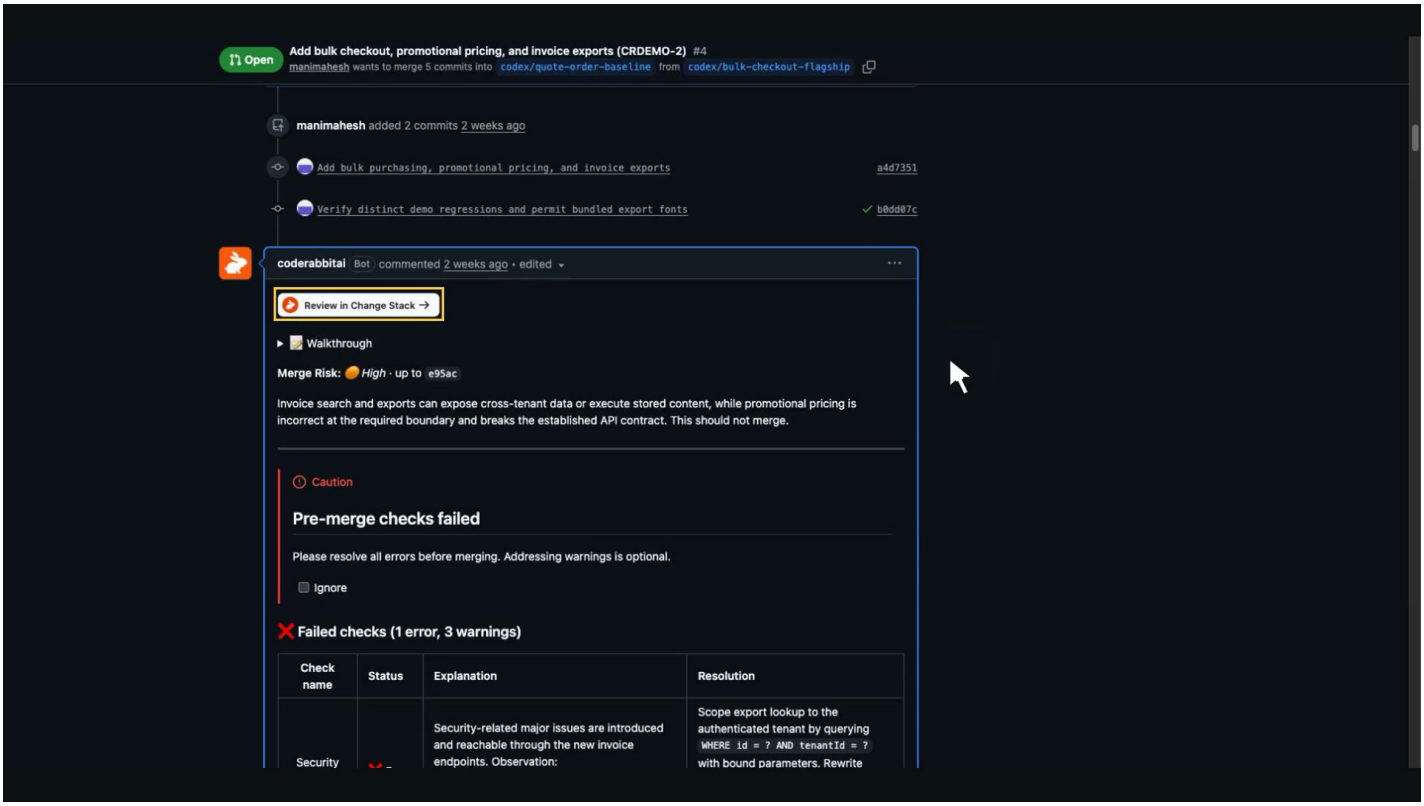
PR #4: SQL finding

PR #4: Change Stack

SAY

Let's open the pull request and look at the review comment in context. From here, I'll choose review in Change Stack.

SEE / VIDEO FRAME AT 01:08



OPEN CHANGE STACK / STEP 2 OF 2 / 01:11-01:20

Read the related layers

DO

In Change Stack, read the left layer rail and select the invoice-search/export layer.

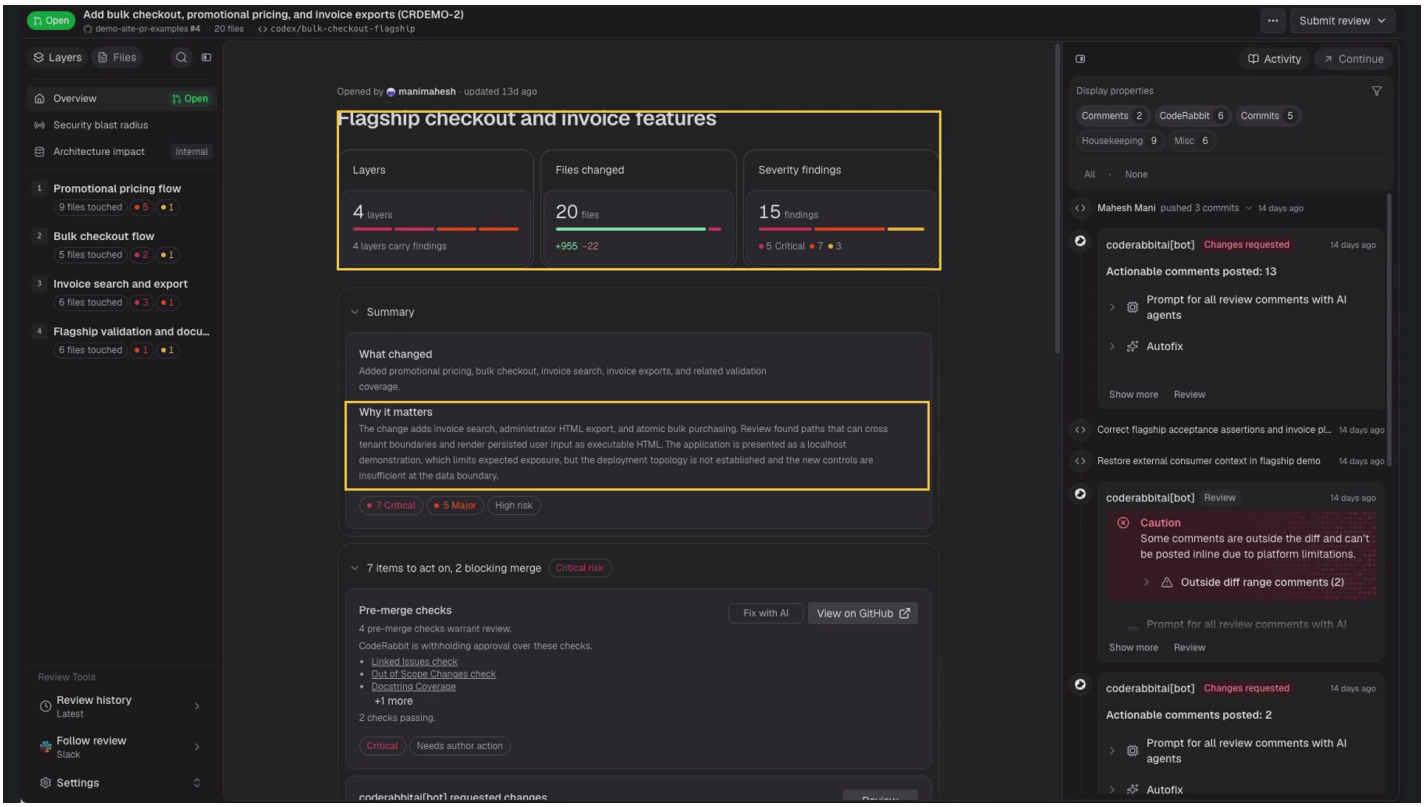
Inspect the grouped changes before opening the impact view.

PR #4: Change Stack

SAY

Change Stack organizes the PR into related layers. That lets me review the invoice changes together and see how they fit into the rest of the pull request.

SEE / VIDEO FRAME AT 01:17



EXPLORE BLAST RADIUS / STEP 1 OF 4 / 01:21-01:30

Open the security graph

DO

Click Security Blast Radius in the left rail. Select the SQL injection concern.

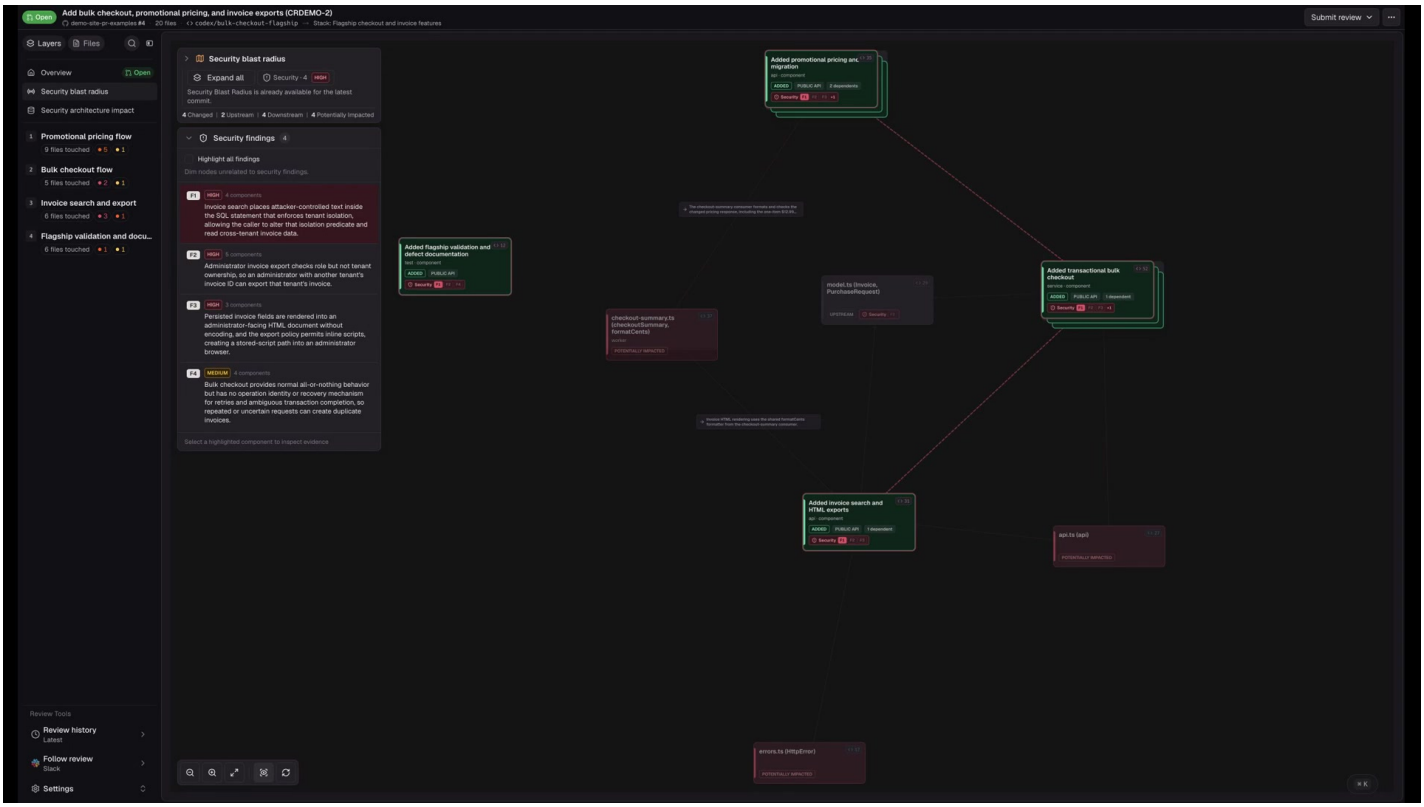
Keep the graph legend and changed components visible.

PR #4: Security Blast Radius

SAY

Now I'll open security blast radius and select the SQL injection concern. The graph shows the changed components and their relationships.

SEE / VIDEO FRAME AT 01:27



EXPLORE BLAST RADIUS / STEP 2 OF 4 / 01:30-01:39

Inspect dependency and consumer relationships

DO

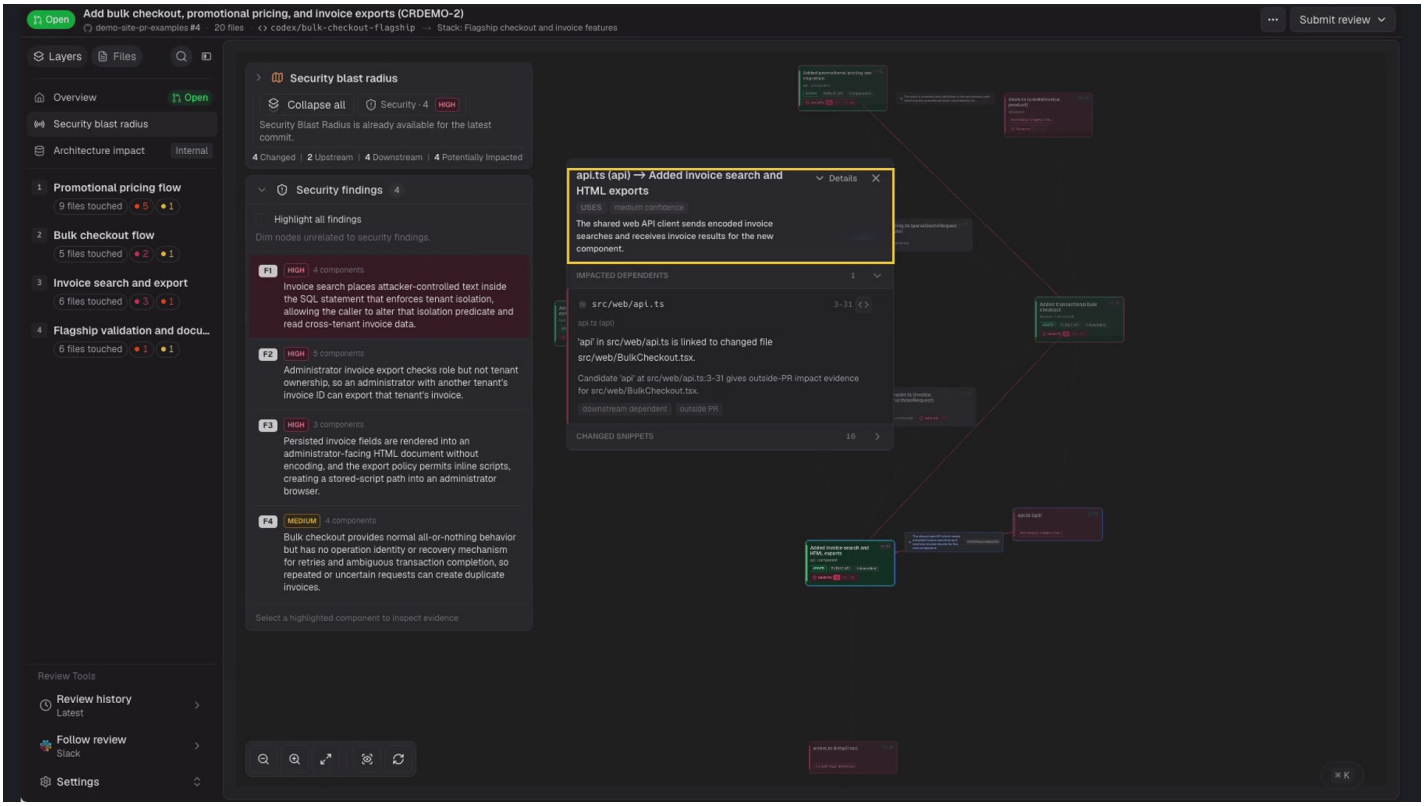
Select the shared data model relationship for invoice search/export.

Read its evidence, then select the web-client consumer relationship and inspect its request/result context.

SAY

For invoice search and export, I can inspect the shared data model. This code depends on and the web client that sends searches and receives results.

SEE / VIDEO FRAME AT 01:37



EXPLORE BLAST RADIUS / STEP 3 OF 4 / 01:40-01:52

Read supporting evidence

DO

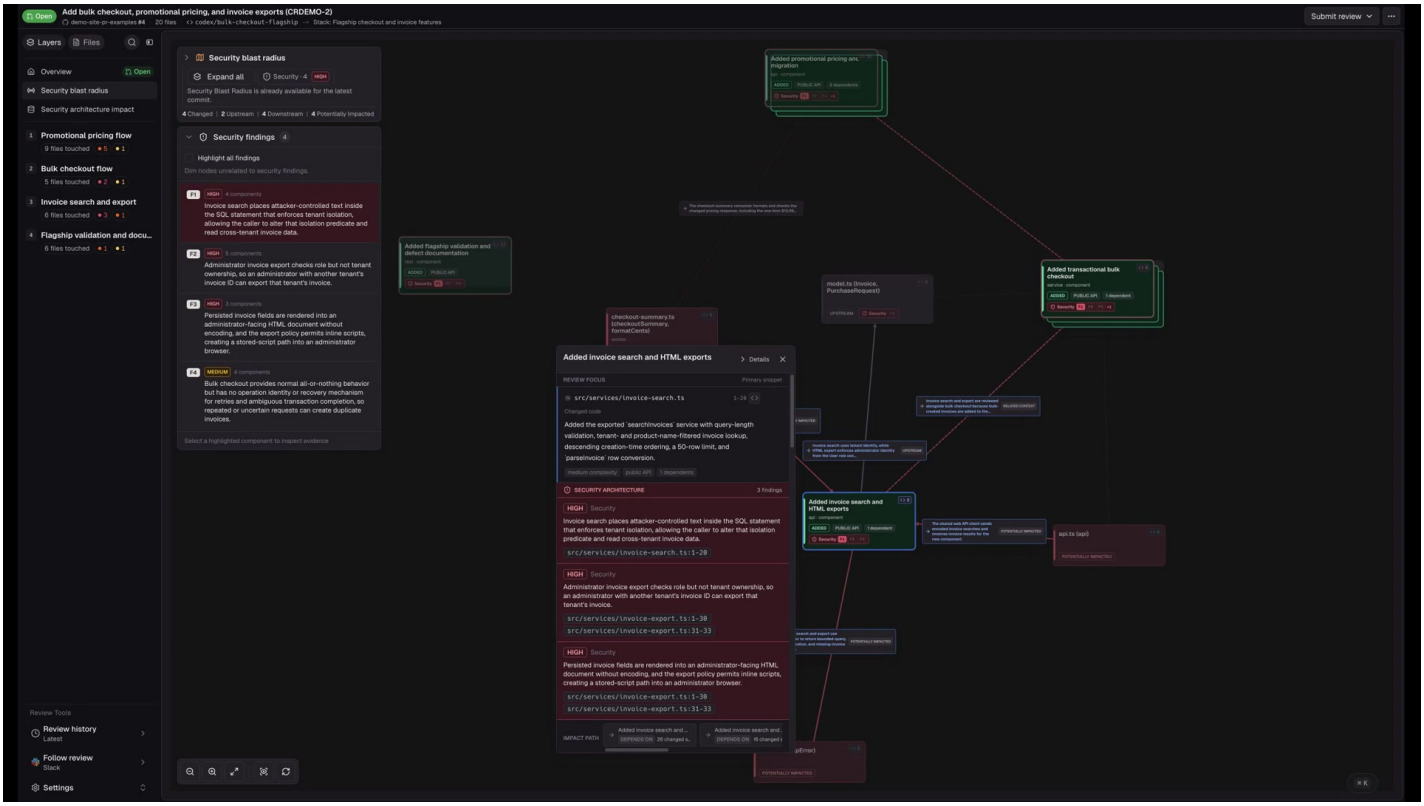
Open the relationship's supporting code evidence.

Scroll inside the evidence panel and distinguish connected components from confirmed vulnerabilities.

SAY

Those connections help me decide where else to look. They don't mean every connected component is vulnerable. I can open the supporting evidence and check what each relationship actually involves.

SEE / VIDEO FRAME AT 01:48



EXPLORE BLAST RADIUS / STEP 4 OF 4 / 01:53-02:05

Check the tenant boundary

DO

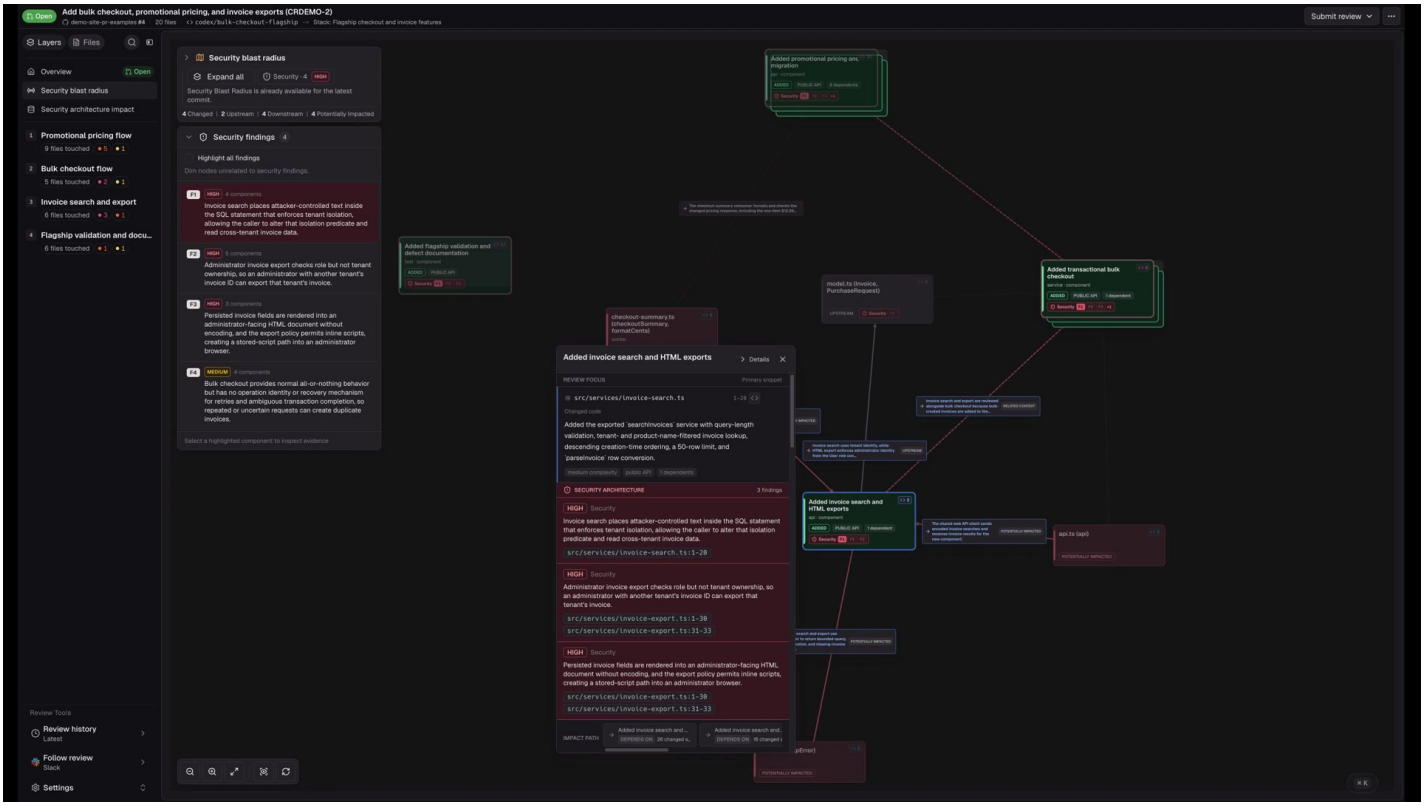
Keep the relevant query path visible.

Trace customer-controlled search text to the SQL command and the tenant boundary before starting the fix.

SAY

In this case, the important question is whether customer controlled search text can alter the database query and cross boundary between tenants. That gives me a specific behavior to check when I review the fix.

SEE / VIDEO FRAME AT 02:01



FIX WITH AI / STEP 1 OF 4 / 02:06-02:19

Select the finding beside its code

DO

Click the affected graph component, then Open in layer.

Open the context panel and select Comments. Scroll to the SQL finding.

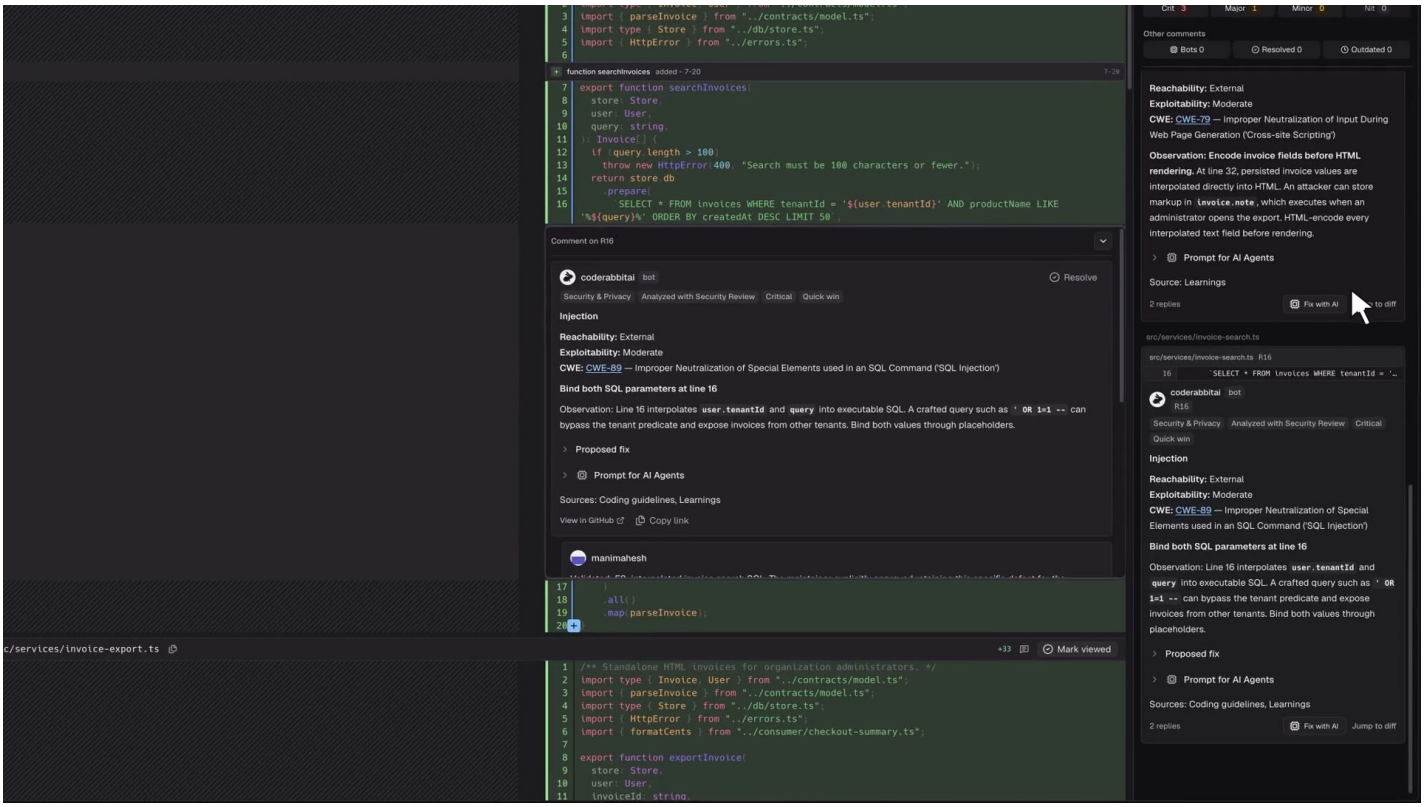
Click Fix with AI for that finding.

PR #4: Security Blast Radius

SAY

Let's address the finding. I'll open the affected component, choose open in layer and return to the review comments beside the code. In the comments panel, I'll select fix with AI for the SQL injection finding.

SEE / VIDEO FRAME AT 02:14



FIX WITH AI / STEP 2 OF 4 / 02:19-02:38

Choose delivery and open the completed run

DO

Review the selected comment and click the next/delivery control.

Select Create a new PR with the fixes. The recorded setup previews this choice.

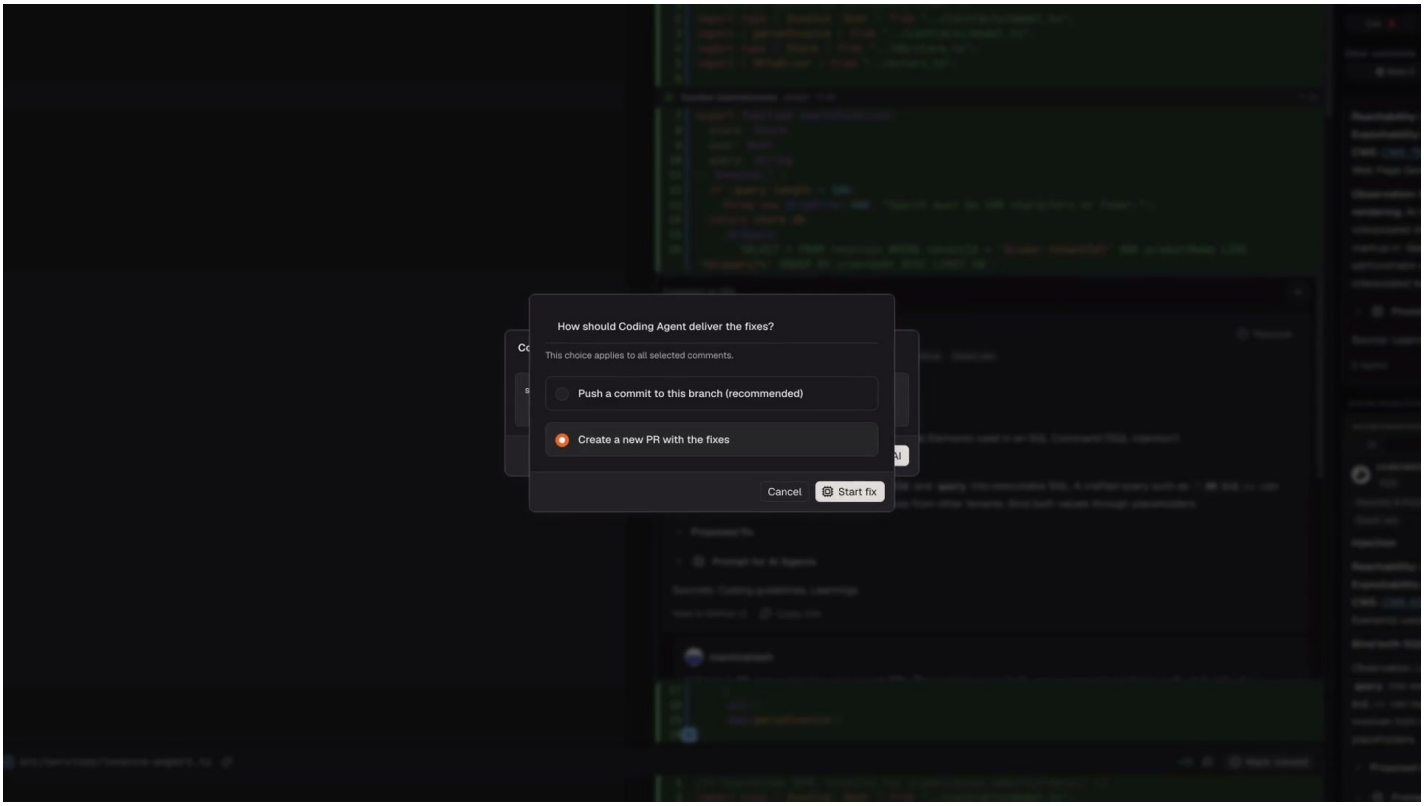
Switch to the completed task in the recording repository to inspect the result.

Completed Engineering Security task

SAY

Then I'll review the selected comment and choose how the agent should deliver the change. I'm choosing a separate pull request so I can inspect the proposed patch before it reaches the original branch. To show the result, let's switch to a complete run of this same fix in our recording repository.

SEE / VIDEO FRAME AT 02:32



FIX WITH AI / STEP 3 OF 4 / 02:39-02:57

Inspect the SQL patch and regressions

DO

Follow the completed task's delivered PR #11 link. On GitHub, click Files changed and select invoice-search.ts.

Scroll to bound tenant/search parameters and literal wildcard handling.

Open the regression test files and inspect injection and tenant-isolation cases.

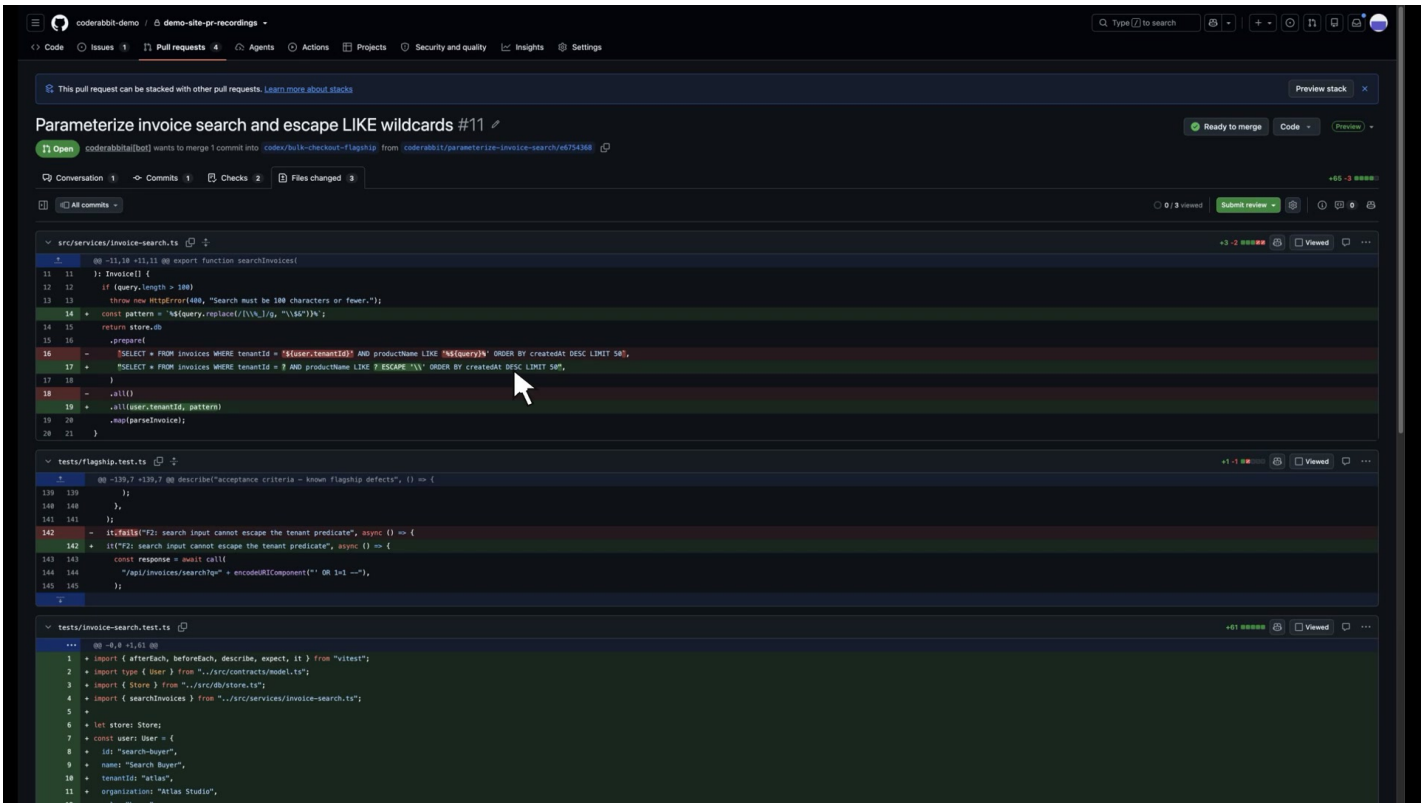
Completed Engineering Security task

PR #11: files changed

SAY

The patch replaces the interpolated SQL with bound parameters for the tenant ID and search pattern. It also escapes wildcard characters, so a search for those characters matches them literally. The agent added regression tests for injection attempts and tenant isolation.

SEE / VIDEO FRAME AT 02:50



FIX WITH AI / STEP 4 OF 4 / 02:57-03:12

Read validation and the delivered PR

DO

Return to the task summary and read the 71-test result and four expected failures.

Open PR #11. Click Files changed to inspect code/tests, then return to Conversation.

Leave the proposed fix open and unmerged.

Completed Engineering Security task

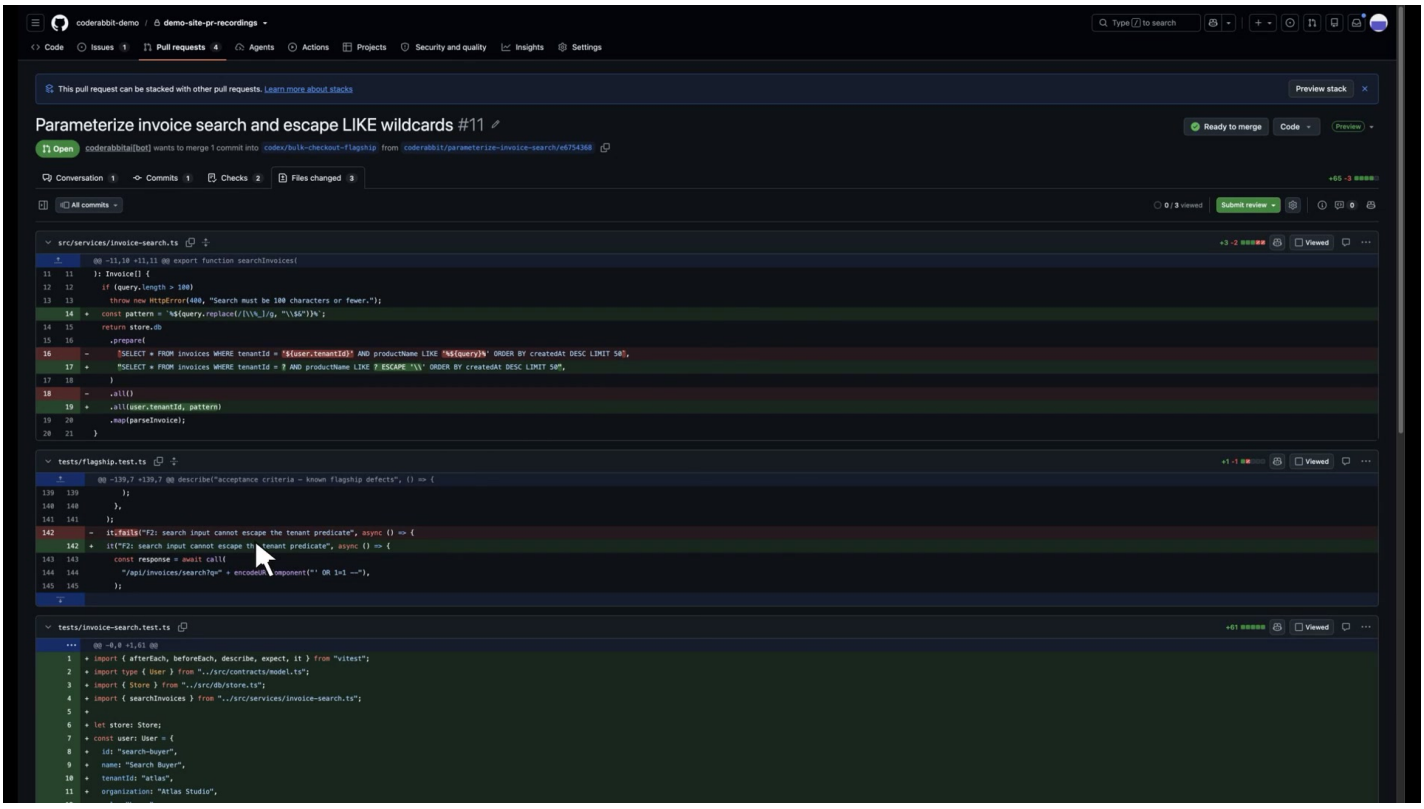
Engineering Security fix: PR #11

PR #11: files changed

SAY

The task reports 71 passing tests with four unrelated and expected failures. Here's the resulting pull request. I can inspect the code and the tests before deciding whether to merge. For this demo, we are leaving it open.

SEE / VIDEO FRAME AT 03:07



REVIEW MERGE READINESS / STEP 1 OF 3 / 03:13-03:21

Return to the original PR

DO

Open flagship PR #4 and scroll to the Conversation merge panel.

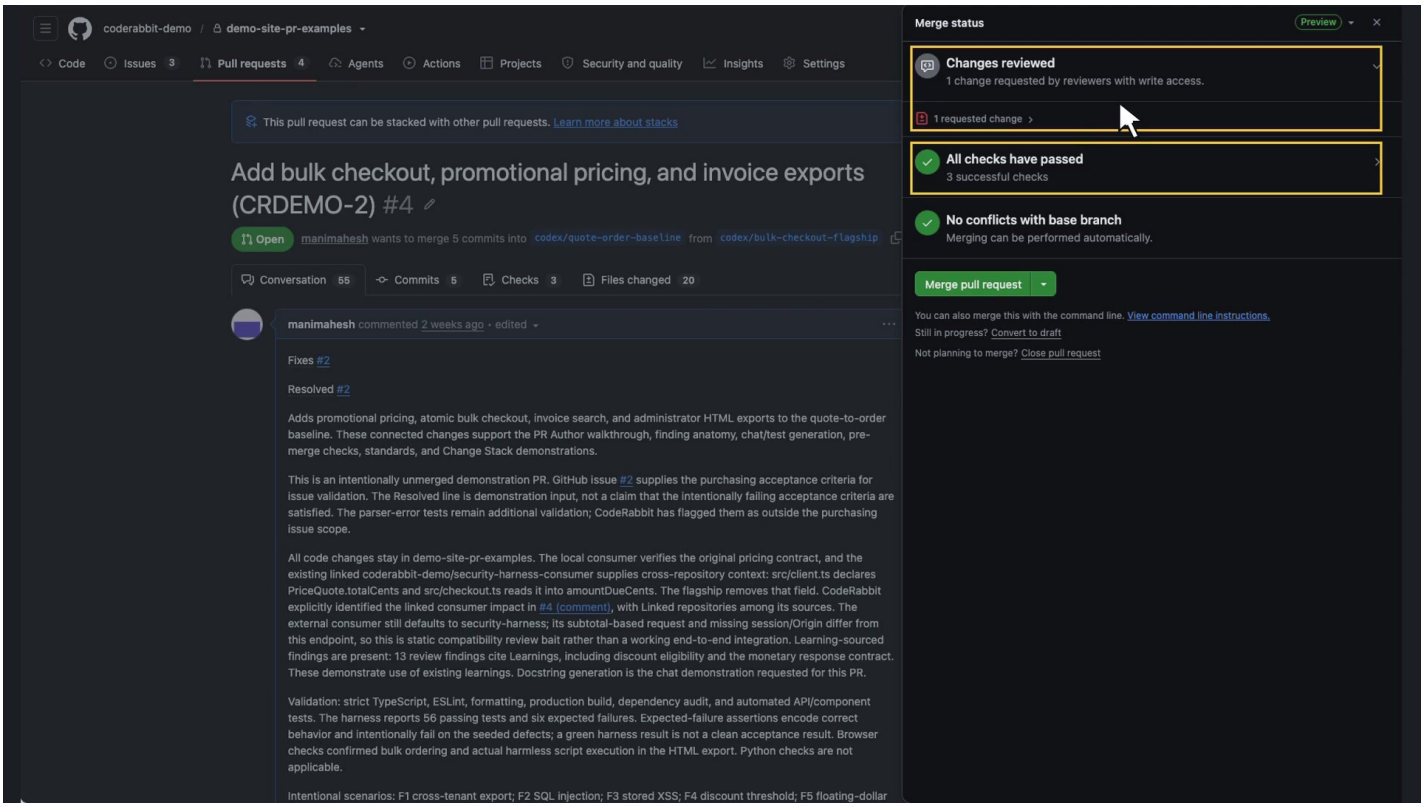
Show Changes requested alongside passed application checks.

Flagship PR #4

SAY

Back on the original pull request, CodeRabbit has requested changes. The application checks passed, but the security concerns still needs to be addressed.

SEE / VIDEO FRAME AT 03:18



REVIEW MERGE READINESS / STEP 2 OF 3 / 03:21-03:33

Read the enforcement state

DO

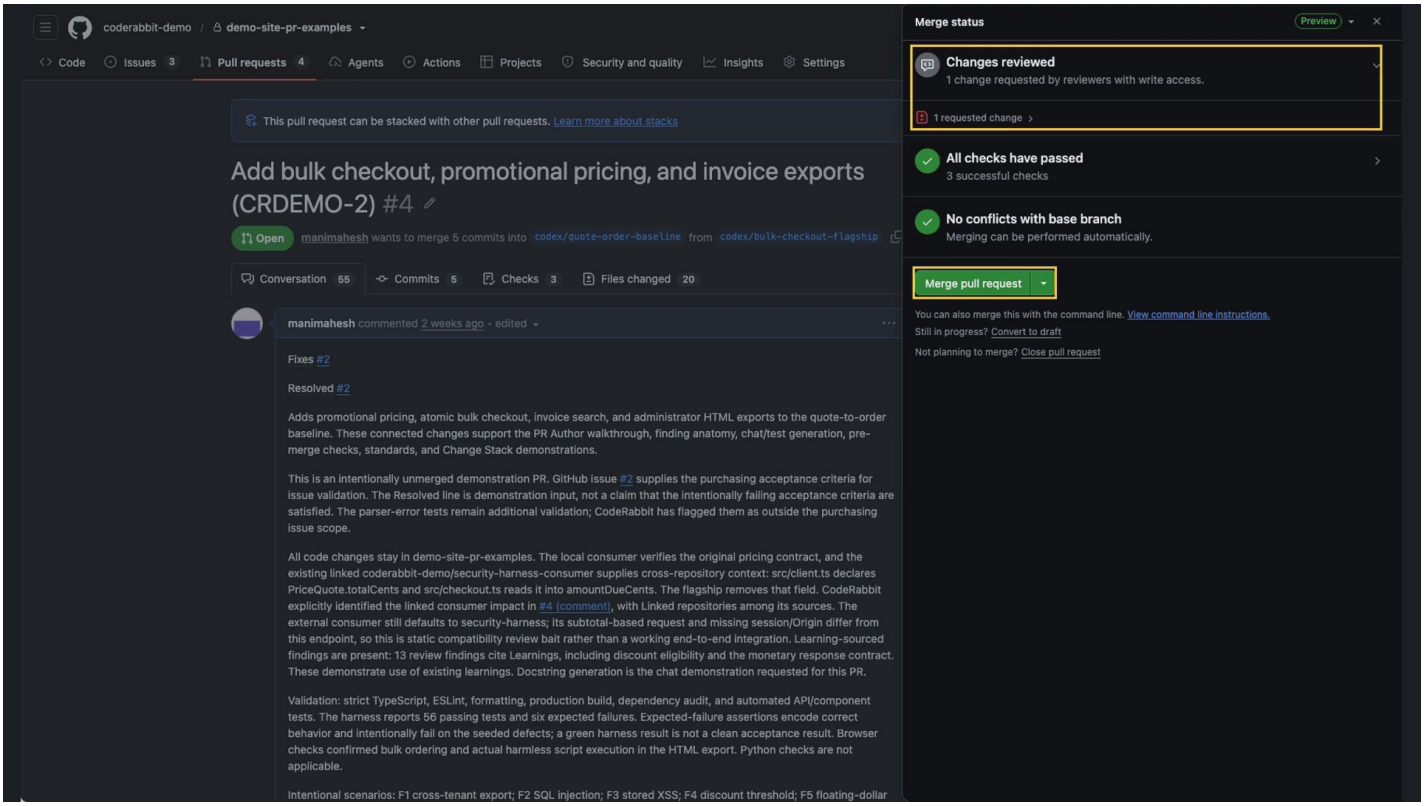
Keep the merge panel visible. Read the absence of branch protection for this demo branch.

Show the available merge control without clicking it; no branch settings are changed.

SAY

This demo branch doesn't have protection enabled, so the request for changes isn't an enforcement block. With branch rules, a team can require the checks and approvals it needs before accepting a change.

SEE / VIDEO FRAME AT 03:29



REVIEW MERGE READINESS / STEP 3 OF 3 / 03:35-03:43

Finish on the review decision

DO

Keep the original PR and proposed fix available while finishing the narration.

Show the closing flow slide. Leave both PRs open.

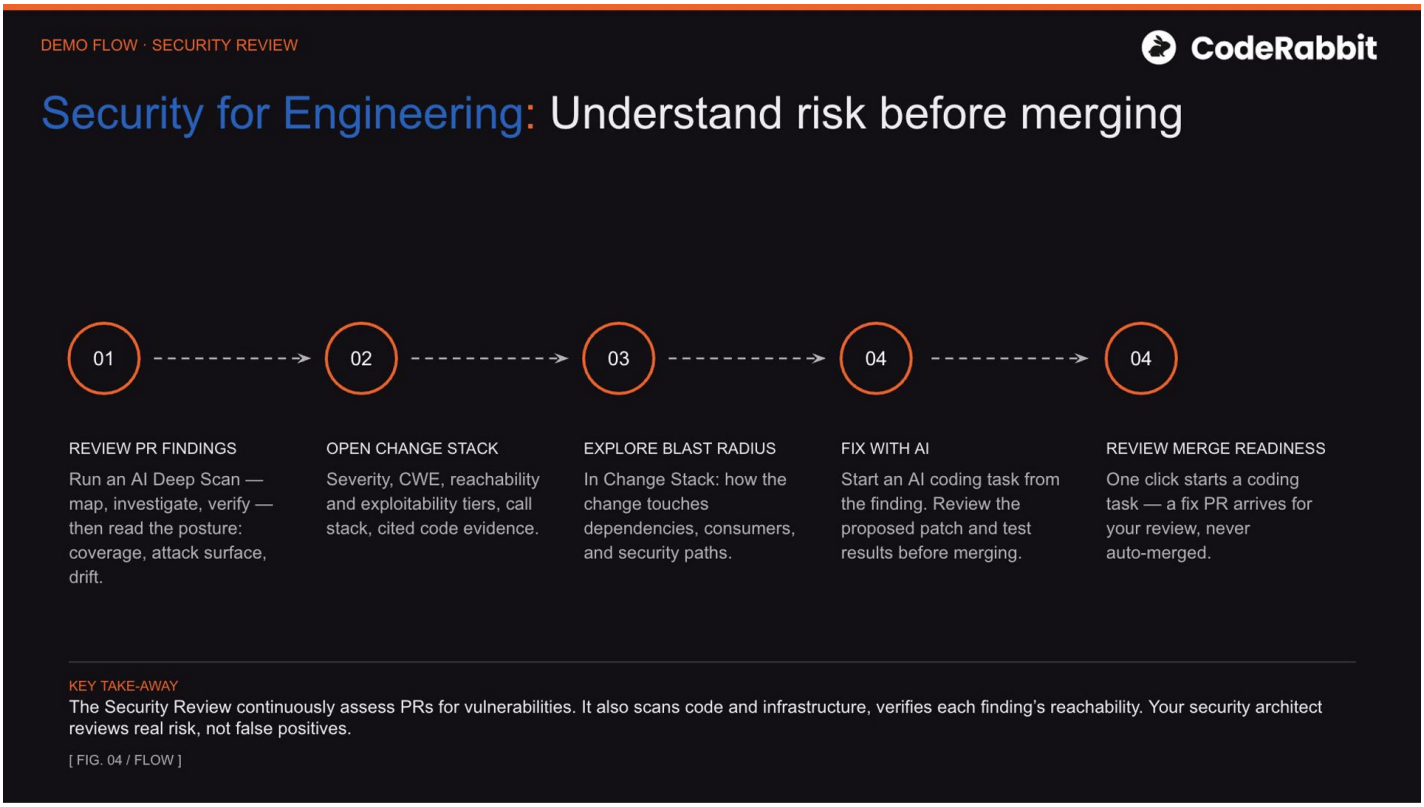
Flagship PR #4

Engineering Security fix: PR #11

SAY

We followed a security finding from the PR to its wider impact, prepared a proposed fix and identified what needs review before merging.

SEE / VIDEO FRAME AT 03:43



Resource directory

Links open the recorded examples. Historical comments and pinned source revisions preserve the relevant context when the live PR has moved on.

Flagship PR #4

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/4

PR #4: Change Stack

https://app.coderabbit.ai/change-stack/coderabbit-demo/demo-site-pr-examples/pull/4/overview

PR #4: Security Blast Radius

https://app.coderabbit.ai/change-stack/coderabbit-demo/demo-site-pr-examples/pull/4/blast-radius

Completed Engineering Security task

https://app.coderabbit.ai/code/tasks/e6754368-4d77-4394-8161-8270bdab18af

Engineering Security fix: PR #11

https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/11

Deep Scan finding: CWE-89

https://app.coderabbit.ai/security/code?owner=coderabbit-demo&repo=demo-site-pr-examples&findingId=5bbe54d1-f97b-48ba-9a95-9cc1300c4cdf

PR #4: SQL finding

https://github.com/coderabbit-demo/demo-site-pr-examples/pull/4#discussion_r3991696811

PR #11: files changed

https://github.com/coderabbit-demo/demo-site-pr-recordings/pull/11/files